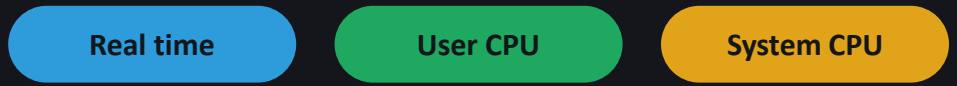
Linux Command Execution Time Profiler

Team Members:
2520090235- Rohita
2520090017- L. Tejasri
2520090007- S. Dikshita



Problem Statement

Developers and sysadmins need to know how long a command takes and what resources it uses, but the built-in tools each show only one slice.

Goal
One tool that runs any command and shows execution time and resource usage, visually.

1. **Scattered tools**
   time, strace and top each report a different piece.

2. **Raw text output**
   Numbers are hard to compare at a glance.

3. **No unified view**
   Time, memory and syscalls are never shown together.

System Design

1
GTK3 GUI
Command entry and Start button

→

2
Worker Thread
pthread keeps the UI responsive

→

3
Tracer (parent)
fork + ptrace PTRACE_SYSCALL

→

4
Target (child)
Runs /bin/sh -c "command"

→

5
Result Store
ProfileResult struct

1
Execution Profile
Summary text + bar chart (Cairo)

2
Syscall Breakdown
Table of syscall name and count

3
Live Memory Graph
Memory timeline curve

Implementation Approach

1

Spawn

Child calls PTRACE_TRACEME, then execs the command through /bin/sh.

2

Trace

PTRACE_SYSCALL stops the child at each syscall; orig_rax identifies it.

3

Measure

CLOCK_MONOTONIC for real time; getrusage for user/sys CPU and peak RSS.

4

Present

g_idle_add updates the text, bar chart and syscall table on the UI thread.

```
pid = fork();
if (pid == 0) {
  ptrace(PTRACE_TRACEME,0,0,0);
  execl("/bin/sh","sh","-c",cmd,NULL);
}
while (1) {
  ptrace(PTRACE_SYSCALL,pid,0,0);
  waitpid(pid, &status, 0);
  if (WIFEXITED(status)) break;
  // read orig_rax, count syscall
}
getrusage(RUSAGE_CHILDREN,&usage);
```

Technologies Used

Language

C

Systems-level code with direct kernel access

GUI

GTK 3

Window, entry, notebook tabs, tree view

Graphics

Cairo

Custom bar chart and memory curve drawing

Concurrency

POSIX threads

Worker thread keeps the UI responsive

Tracing

ptrace

Syscall interception on x86-64 registers

Measurement

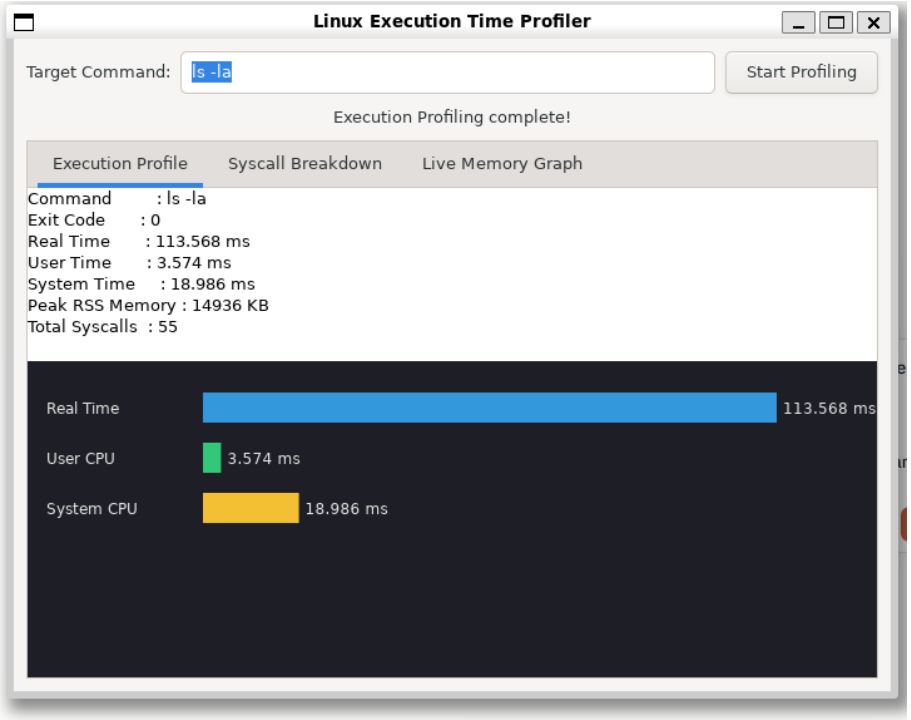
Linux APIs

clock_gettime, getrusage, waitpid

Testing

Scenario	Example command	What it verifies
Quick command	ls -la	Baseline metrics; exit code 0
CPU-bound	gzip on a large file	User CPU time dominates
I/O-heavy	dd / cat on a big file	System time and read/write counts rise
Memory-heavy	program allocating a large array	Peak RSS grows accordingly
Invalid command	nonexistent_cmd	Exit code 127 is reported

Results



113.568 ms Real time

3.574 ms User CPU

18.986 ms System CPU

14,936 KB Peak RSS memory

55 Total syscalls

Impact and Future Scope

Impact

- One window shows time, memory and syscalls together
- Real vs user vs system time makes CPU or I/O bottlenecks easy to spot
- A hands-on way to learn processes, ptrace and system calls

Future Scope

- Sample VmRSS from /proc to fill the live memory graph
- Complete syscall name table and per-call latency
- Export results to CSV/JSON and compare runs

Thank You

Linux Command Execution Time Profiler